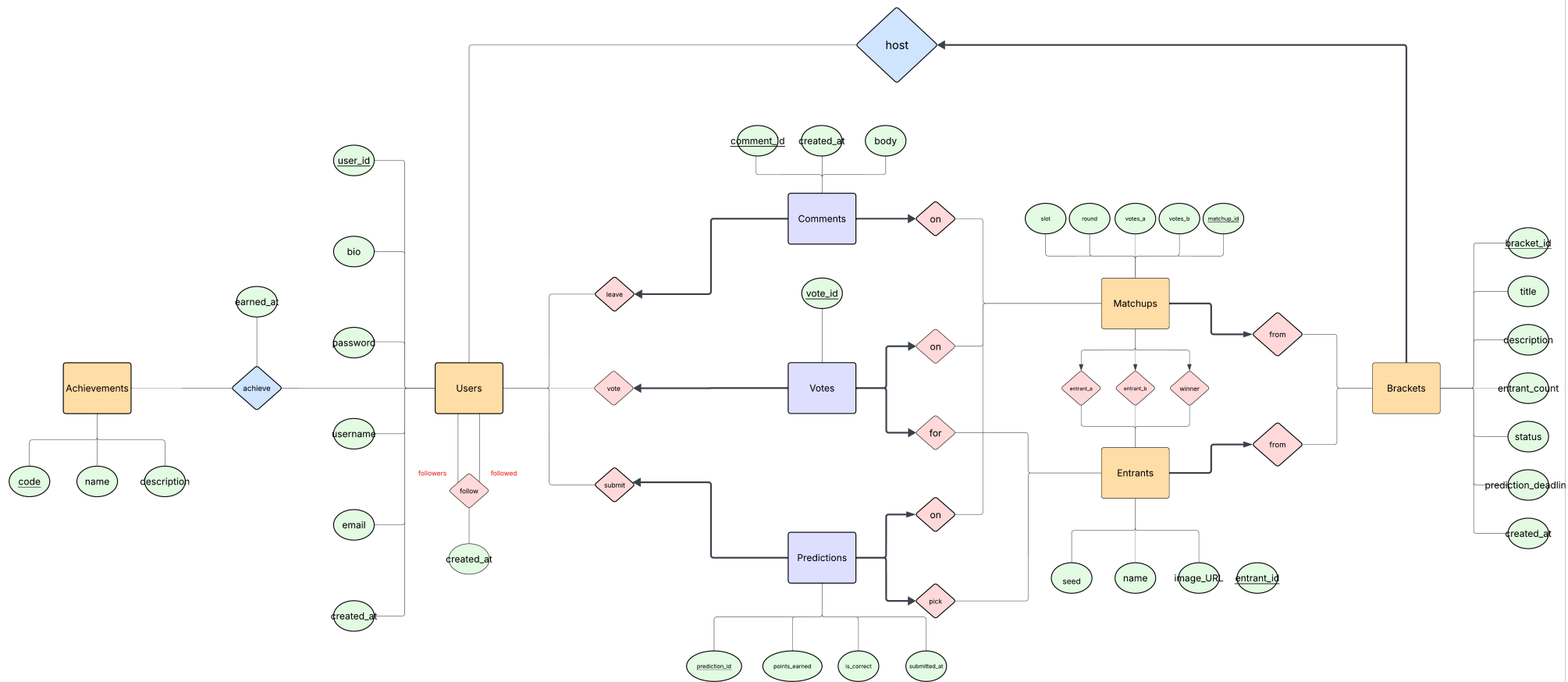


CS 460 Final Project: Phase I

Tan Baydar

1 E-R diagram

The E-R diagram has been added full size to next page.



2 Relational Schema

```
CREATE TABLE Users (  
    user_id      INT AUTO_INCREMENT PRIMARY KEY,  
    username     VARCHAR(50) NOT NULL UNIQUE,  
    email        VARCHAR(255) NOT NULL UNIQUE,  
    password     VARCHAR(255) NOT NULL,  
    bio          TEXT,  
    created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE Brackets (  
    bracket_id   INT AUTO_INCREMENT PRIMARY KEY,  
    host_id      INT NOT NULL,  
    title        VARCHAR(255) NOT NULL,  
    description   TEXT,  
    entrant_count INT NOT NULL,  
    status       ENUM(  
        'draft',  
        'predictions_open',  
        'round_1', 'round_2', 'round_3', 'round_4', 'round_5',  
        'completed'  
    ) NOT NULL DEFAULT 'predictions_open',  
    prediction_deadline DATETIME,  
    created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT chk_entrant_count CHECK (entrant_count IN (4,8,16,32)),  
    CONSTRAINT fk_brackets_host FOREIGN KEY (host_id) REFERENCES Users(user_id)  
);  
  
CREATE TABLE Entrants (  
    entrant_id   INT AUTO_INCREMENT PRIMARY KEY,  
    bracket_id   INT NOT NULL,  
    seed         INT NOT NULL,  
    name         VARCHAR(255) NOT NULL,  
    image_URL    VARCHAR(255),  
    CONSTRAINT fk_entrants_bracket FOREIGN KEY (bracket_id) REFERENCES Brackets(bracket_id),  
    CONSTRAINT uq_entrants_seed    UNIQUE (bracket_id, seed)  
);  
  
CREATE TABLE Matchups (  
    matchup_id   INT AUTO_INCREMENT PRIMARY KEY,  
    bracket_id   INT NOT NULL,  
    round        INT NOT NULL,  
    slot         INT NOT NULL,  
    entrant_a_id INT,  
    entrant_b_id INT,  
    winner_entrant_id INT,  
    votes_a      INT NOT NULL DEFAULT 0,  
    votes_b      INT NOT NULL DEFAULT 0,  
    CONSTRAINT chk_entrant1_entrant2 CHECK (entrant_a_id <> entrant_b_id),  
    CONSTRAINT chk_winner_entrants CHECK (winner_entrant_id IN (entrant_a_id, entrant_b_id)),  
    CONSTRAINT fk_matchups_bracket FOREIGN KEY (bracket_id) REFERENCES Brackets(bracket_id),  
    CONSTRAINT fk_matchups_a FOREIGN KEY (entrant_a_id) REFERENCES Entrants(entrant_id),  
    CONSTRAINT fk_matchups_b FOREIGN KEY (entrant_b_id) REFERENCES Entrants(entrant_id),  
    CONSTRAINT fk_matchups_winner FOREIGN KEY (winner_entrant_id) REFERENCES Entrants(entrant_id),  
    CONSTRAINT uq_matchups_slot    UNIQUE (bracket_id, round, slot)  
);
```

```

CREATE TABLE Achievements (
    code VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT NOT NULL
);

CREATE TABLE Predictions (
    prediction_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    matchup_id INT NOT NULL,
    entrant_id INT NOT NULL,
    is_correct BOOLEAN,
    points_earned INT NOT NULL DEFAULT 0,
    submitted_at DATETIME NOT NULL,
    CONSTRAINT uq_predictions_matchup UNIQUE (user_id, matchup_id),
    CONSTRAINT fk_predictions_user FOREIGN KEY (user_id) REFERENCES Users(user_id),
    CONSTRAINT fk_predictions_matchup FOREIGN KEY (matchup_id) REFERENCES Matchups(matchup_id),
    CONSTRAINT fk_predictions_entrant FOREIGN KEY (entrant_id) REFERENCES Entrants(entrant_id)
);

CREATE TABLE Votes (
    vote_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    matchup_id INT NOT NULL,
    entrant_id INT NOT NULL,
    CONSTRAINT fk_votes_user FOREIGN KEY (user_id) REFERENCES Users(user_id),
    CONSTRAINT fk_votes_matchup FOREIGN KEY (matchup_id) REFERENCES Matchups(matchup_id),
    CONSTRAINT fk_votes_entrant FOREIGN KEY (entrant_id) REFERENCES Entrants(entrant_id),
    CONSTRAINT uq_votes_matchup UNIQUE (user_id, matchup_id)
);

CREATE TABLE user_achievements (
    user_id INT NOT NULL,
    code VARCHAR(50) NOT NULL,
    earned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, code),
    CONSTRAINT fk_user_achievements_user FOREIGN KEY (user_id) REFERENCES Users(user_id),
    CONSTRAINT fk_user_achievements_achievement FOREIGN KEY (code) REFERENCES Achievements(code)
);

CREATE TABLE Follows (
    follower_id INT NOT NULL,
    followed_id INT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY(follower_id, followed_id),
    CONSTRAINT chk_followers CHECK (follower_id <> followed_id),
    CONSTRAINT fk_follows_follower FOREIGN KEY (follower_id) REFERENCES Users(user_id),
    CONSTRAINT fk_follows_followed FOREIGN KEY (followed_id) REFERENCES Users(user_id)
);

CREATE TABLE Comments (
    comment_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    matchup_id INT NOT NULL,
    body TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_comments_user FOREIGN KEY (user_id) REFERENCES Users(user_id),
    CONSTRAINT fk_comments_matchup FOREIGN KEY (matchup_id) REFERENCES Matchups(matchup_id)
);

```

3 Guardrails

3.1 Triggers

1

```
CREATE TRIGGER predictionsOpen BEFORE INSERT ON Predictions
REFERENCING NEW as newTuple
FOR EACH ROW
BEGIN
  IF (SELECT B.status
      FROM Matchups M, Brackets B
      WHERE M.matchup_id = newTuple.matchup_id
      AND M.bracket_id = B.bracket_id) <> 'predictions_open'
  THEN
    RAISE_APPLICATION_ERROR (-200011, 'Predictions may only be submitted while the bracket is in the predictions
      are open state.');
```

2

```
CREATE TRIGGER predictionsRoundOne BEFORE INSERT ON Predictions
REFERENCING NEW as newTuple
FOR EACH ROW
BEGIN
  IF (SELECT M.round
      FROM Matchups M
      WHERE M.matchup_id = newTuple.matchup_id) <> 1
  THEN
    RAISE_APPLICATION_ERROR (-20003,
      'Any logged-in user may submit one prediction per Round 1 matchup.');
```

3

```
CREATE TRIGGER voteCurrentRound BEFORE INSERT ON Votes
REFERENCING NEW as newTuple
FOR EACH ROW
BEGIN
  IF NOT EXISTS (SELECT *
      FROM Matchups M, Brackets B
      WHERE M.matchup_id = newTuple.matchup_id
      AND M.bracket_id = B.bracket_id
      AND B.status = 'round_' || M.round)
  THEN
    RAISE_APPLICATION_ERROR (-20002, 'Votes may only be cast on a matchup whose round equals the bracket's
      current round.');
```

¹In the documentation, I followed that valid manual error codes were determined in the range “-20000 to -20999” via IBM Db2 RAISE APPLICATION ERROR documentation: <https://www.ibm.com/docs/en/db2/11.1.0?topic=plsql-raise-application-error>.

4

```
CREATE TRIGGER awardBracketMaker AFTER INSERT ON Brackets
REFERENCING NEW as newTuple
FOR EACH ROW
BEGIN
  IF (SELECT COUNT(*)
      FROM Brackets B
      WHERE B.host_id = newTuple.host_id) = 1
  THEN
    INSERT INTO user_achievements (user_id, code)
    VALUES (newTuple.host_id, 'bracket_maker');
  END IF;
END;
```

5

```
CREATE TRIGGER awardLockedIn AFTER INSERT ON Predictions
REFERENCING NEW as newTuple
FOR EACH ROW
BEGIN
  IF (SELECT COUNT(*)
      FROM Predictions P
      WHERE P.user_id = newTuple.user_id) = 10
  THEN
    INSERT INTO user_achievements (user_id, code)
    VALUES (newTuple.user_id, 'locked_in');
  END IF;
END;
```

6

```
CREATE TRIGGER commentRoundOne BEFORE INSERT ON Comments
REFERENCING NEW as newTuple
FOR EACH ROW
BEGIN
  IF (SELECT M.round
      FROM Matchups M
      WHERE M.matchup_id = newTuple.matchup_id) <> 1
  THEN
    RAISE_APPLICATION_ERROR (-20006,
        'Comments may only be posted on Round 1 matchups.');
```

```
  END IF;
END;
```

Assertions

```
CREATE ASSERTION entrantCountExact
CHECK (NOT EXISTS
  (SELECT *
   FROM Brackets B
   WHERE B.entrant_count <>
        (SELECT COUNT(*)
         FROM Entrants E
         WHERE E.bracket_id = B.bracket_id)));

CREATE ASSERTION matchupCountExact
CHECK (NOT EXISTS
  (SELECT *
   FROM Brackets B
   WHERE B.entrant_count - 1 <>
        (SELECT COUNT(*)
         FROM Matchups M
         WHERE M.bracket_id = B.bracket_id)));

CREATE ASSERTION roundSizeExact
CHECK (NOT EXISTS
  (SELECT *
   FROM Matchups M, Brackets B
   WHERE M.bracket_id = B.bracket_id
        AND (SELECT COUNT(*)
              FROM Matchups M2
              WHERE M2.bracket_id = M.bracket_id
                 AND M2.round = M.round)
        <> B.entrant_count / POWER2(2, M.round)));
```

Assumptions

Assumptions:

The schema does not check or enforce that a prediction or a vote is on one of the two entrants of the related matchup. This is an assumption based on the upcoming buttons that will be implemented in the second phase of the project.

We assume seeds are numbered 1 through `entrant_count` as $1, 2, \dots, \text{entrant_count}$) in a counting order, and that slot k of Round 1 holds the matchup between seeds $2k - 1$ and $2k$.

We assume the `bracket_maker` and `locked_in` rows in the Achievements table will be handled, initiated, and logged by the implementation in Phase II.

²In order to satisfy the requirement: “Each bracket contains exactly $(\text{entrant_count}-1)$ matchups arranged across $\log_2(\text{entrant_count})$ rounds,” I used the documentation of MySQL to find a log/power operator. Found it in MySQL Mathematical Functions: <https://dev.mysql.com/doc/refman/9.7/en/mathematical-functions.html>.